REPORTE DE PRÁCTICA

Práctica 3. Interacción Avanzada con React Native

Fusión 2048: juego tipo 2048 con React Native, Expo y Expo Router



Integrantes del equipo

Axel Cerdeira

Carlos Guerrero

Alberto Nuño

10 de octubre de 2026

Índice

1. Objetivo 3
1.1 Objetivos específicos 3
2. Introducción 4
3. Materiales y recursos 5
3.1 Software y bibliotecas 5
3.2 Equipo 5
3.3 Recursos gráficos 5
4. Desarrollo experimental 6
4.1 Creación y configuración del proyecto 6
4.2 Arquitectura y estructura del proyecto 6
4.3 Navegación con Expo Router 7
4.4 Lógica del juego y puntuación 8
4.5 Mecánicas especiales: Mover y Eliminar 9
4.6 Gestos, animaciones y retroalimentación 10
4.7 Estilos reutilizables e identidad visual 11
4.8 Imágenes de los recursos utilizados 12
4.9 Explicación y justificación de los gráficos de las celdas 13
4.10 Capturas de las pantallas de la aplicación 15
4.11 Ejecución en un dispositivo físico con Expo Go 18
5. Análisis de resultados 19
5.1 Pruebas realizadas 19
5.2 Problemas encontrados y soluciones 20
6. Mejoras a futuro 22
6.1 Tablero de puntuaciones en línea mediante una API web segura 22
6.2 Otras mejoras 24
7. Conclusiones 25
7.1 Ventajas de React Native y Expo 25
7.2 Dificultades 25
7.3 Contraste con motores de videojuegos: objetos de juego y ECS 25
8. Referencias 27

1. Objetivo

Desarrollar una aplicación móvil interactiva con **React Native** y **Expo**, navegada con **Expo Router**, que implemente un juego basado en la mecánica de **2048** (fusión de celdas iguales y contiguas), integrando manejo de estado, gestos táctiles, animaciones y una identidad visual consistente, y que funcione en un dispositivo físico mediante **Expo Go**.

1.1 Objetivos específicos

- Construir las pantallas de presentación, juego, juego terminado (Game Over) y créditos usando rutas de archivo de Expo Router.
- Programar la lógica de deslizamiento y fusión del 2048, con una puntuación que aumente en cada acción correcta.
- Representar el tipo y el nivel de "poder" de cada celda con un gráfico (emoji temático + color + valor).
- Agregar dos mecánicas especiales: **Mover** (llevar una ficha a la casilla elegida o intercambiarla con otra) y **Eliminar** (quitar una ficha específica).
- Definir estilos reutilizables (tokens de color, tipografía, espaciado y componentes) acordes a un entretenimiento sano, divertido y corporativo.
- Verificar el funcionamiento con pruebas automatizadas, análisis estático y ejecución en Expo Go.

2. Introducción

React Native es un framework de código abierto que permite construir aplicaciones móviles nativas para iOS y Android escribiendo componentes de React en JavaScript o TypeScript. A diferencia de una página web incrustada, los componentes como `View`, `Text` o `Pressable` se traducen a vistas nativas de cada plataforma, por lo que la interfaz conserva el rendimiento y el comportamiento esperados en un teléfono.

Expo es una plataforma construida sobre React Native que agrupa un SDK de módulos listos para usar (vibración háptica, constantes del dispositivo, almacenamiento, etc.), herramientas de línea de comandos y servicios de compilación. Su aplicación **Expo Go** permite ejecutar un proyecto en un teléfono real simplemente escaneando un código QR, sin compilar código nativo, siempre que el proyecto use únicamente los módulos nativos que Expo Go incluye. Esta práctica usa **Expo SDK 57** con React Native 0.86.

Expo Router implementa navegación basada en archivos: cada archivo dentro de `src/app/` es una pantalla y los archivos `_layout.tsx` definen el tipo de navegador (pila, pestañas, etc.). Esto simplifica la estructura, permite enlaces profundos y paso de parámetros entre pantallas.

El juego **2048**, creado por Gabriele Cirulli en 2014, se juega en un tablero de 4×4. Al deslizar en una dirección, todas las fichas se desplazan; cuando dos fichas con el mismo valor chocan se fusionan en una sola con el doble de valor. El objetivo es formar la ficha 2048. Pertenece a la familia de juegos de emparejamiento de fichas (tile-matching), igual que los juegos tipo Match-3.

Para la interacción, React Native ofrece el **sistema de respondedores de gestos** (con `PanResponder` para arrastres) y la API **Animated**, que puede delegar animaciones de transformación al hilo nativo (`useNativeDriver`) para mantener la fluidez. El estado del juego se maneja con el hook `useReducer` de React, que centraliza todas las reglas en una función pura.

3. Materiales y recursos

3.1 Software y bibliotecas

Tabla 1. Software, bibliotecas y versiones utilizadas

Recurso	Versión	Uso en el proyecto
Node.js / npm	24.19.0 / 11.17.0	Entorno de ejecución y gestor de paquetes
Expo SDK (expo)	57.0.27	Plataforma base, CLI y módulos
React Native	0.86.3	Componentes nativos de interfaz
React	19.2.3	Componentes, hooks y manejo de estado
Expo Router	57.0.25	Navegación por archivos (Stack)
react-native-screens / safe-area-context	4.26 / 5.7	Pantallas nativas y áreas seguras (notch, barra inferior)
expo-haptics	57.0.3	Vibración al fusionar, usar poderes y terminar
@react-native-async-storage/async-storage	2.2.0	Guardar el récord en el teléfono
expo-status-bar, expo-constants, expo-linking	57.x	Barra de estado, versión de la app, enlaces
TypeScript	6.0	Tipado estático
ESLint + eslint-config-expo	9 / 57	Análisis estático (incluye reglas del React Compiler)
react-native-web / react-dom	0.21 / 19.2.3	Ejecución en navegador para pruebas
Expo Go	SDK 57	Ejecución en el dispositivo físico
Git	—	Control de versiones

3.2 Equipo

- Computadora con Windows 11 Pro (desarrollo, servidor Metro y pruebas en navegador).
- Teléfono inteligente iOS o Android con la app **Expo Go** (SDK 57), conectado a la misma red Wi-Fi.

3.3 Recursos gráficos

La aplicación no usa archivos de imagen para el juego: las fichas se dibujan con emojis Unicode del sistema sobre fondos de color definidos en código. Los únicos archivos gráficos son los íconos de la app (ícono principal, ícono adaptativo de Android, ícono monocromático, pantalla de carga y favicon), generados a partir de la misma paleta. Sus imágenes se muestran en la sección 4.8.

4. Desarrollo experimental

4.1 Creación y configuración del proyecto

El proyecto se creó con la plantilla de TypeScript de Expo y se agregaron Expo Router y los módulos necesarios con `npx expo install`, que elige automáticamente las versiones compatibles con el SDK instalado. Antes de agregar cualquier biblioteca se revisó la lista de módulos nativos incluidos en Expo Go (`bundledNativeModules.json`), para no requerir una compilación de desarrollo.

Comandos de creación e instalación

```
npx create-expo-app@latest . --template blank-typescript
npx expo install expo-router react-native-safe-area-context react-native-screens \
                 expo-linking expo-constants expo-status-bar expo-haptics
npx expo install @react-native-async-storage/async-storage react-dom react-native-web
```

Después se configuró el punto de entrada de Expo Router en `package.json` (`"main": "expo-router/entry"`), se definió el esquema de enlaces `fusion2048`, las rutas tipadas (`typedRoutes`) y el nombre de la app en `app.json`, y el alias `@/*` → `src/*` en `tsconfig.json` para importar módulos sin rutas relativas largas.

4.2 Arquitectura y estructura del proyecto

El código se organizó en capas: las pantallas solo componen la interfaz; los componentes reutilizables y el tema definen la apariencia; los hooks manejan el estado y la entrada; y la lógica del juego vive en funciones puras de TypeScript, sin dependencia de React, lo que permite probarla de forma aislada.

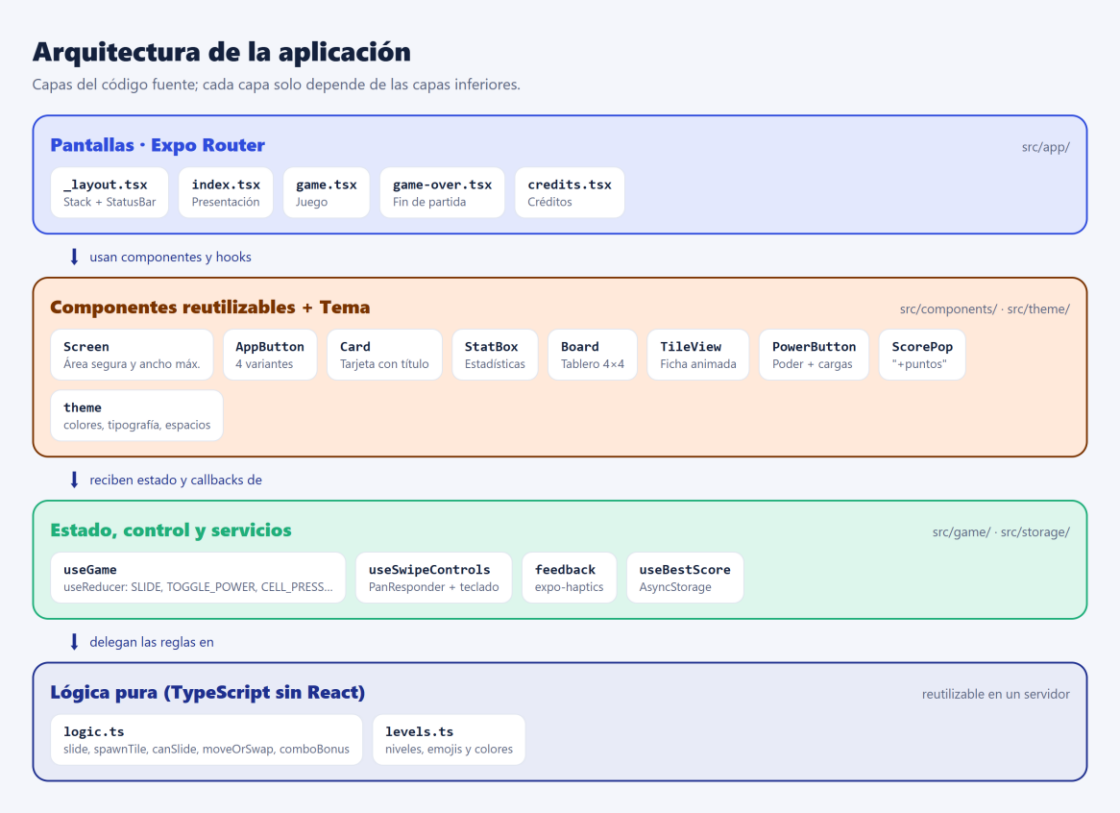


Figura 1. Arquitectura por capas de la aplicación.

Estructura de carpetas

```
src/
  app/         _layout.tsx · index.tsx · game.tsx · game-over.tsx · credits.tsx
  components/  AppButton · Board · Card · PowerButton · Screen · ScorePop · StatBox · TileView
  constants/   app.ts        (textos de la app y créditos)
  game/        logic.ts · levels.ts · useGame.ts · useControls.ts · feedback.ts
  storage/     bestScore.ts  (récord con AsyncStorage)
  theme/       index.ts      (colores, tipografía, espaciado, sombras)
  utils/       confirm.ts    (diálogo de confirmación multiplataforma)
```

4.3 Navegación con Expo Router

El archivo `src/app/_layout.tsx` define un navegador de pila (`Stack`) sin encabezado nativo, con el fondo de la app y transiciones por pantalla. En la pantalla de juego se desactiva el gesto de "atrás" de iOS (`gestureEnabled: false`) para que no se confunda con los deslizamientos horizontales del tablero; en Android, el botón físico de atrás se intercepta con `BackHandler` y pide confirmación antes de abandonar una partida.

src/app/_layout.tsx (extracto)

```
<Stack screenOptions={{ headerShown: false, contentStyle: { backgroundColor: colors.background },
                        animation: 'slide_from_right' }}>
  <Stack.Screen name="index" />
  <Stack.Screen name="game" options={{ gestureEnabled: false, animation: 'fade' }} />
  <Stack.Screen name="game-over" options={{ gestureEnabled: false, animation: 'fade_from_bottom' }} />
  <Stack.Screen name="credits" />
</Stack>
```

Al terminar la partida, la pantalla de juego se **reemplaza** por la de Game Over con `router.replace`, enviando la puntuación, el récord, la ficha máxima, los movimientos y los poderes usados como parámetros; la pantalla final los lee con `useLocalSearchParams`. Así el usuario no puede regresar a una partida ya terminada. Desde Game Over, "Inicio" usa `router.dismissTo('/')` para volver a la presentación cerrando las pantallas intermedias.

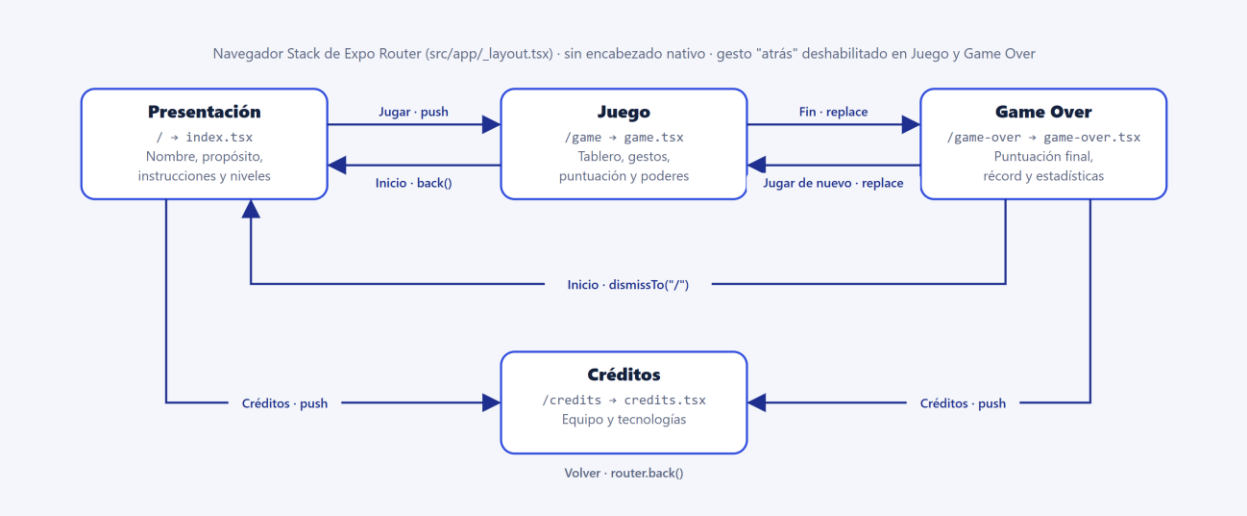


Figura 2. Flujo de navegación entre las cuatro pantallas.

4.4 Lógica del juego y puntuación

Cada ficha se modela como un objeto `{ id, value, row, col }`. El identificador estable permite que React conserve el mismo componente visual mientras la ficha se desplaza, lo que hace posible animar su movimiento. La función `slide` recorre cada fila o columna en el orden de la dirección elegida, compacta las fichas hacia el borde y fusiona pares iguales consecutivos; cada ficha puede fusionarse **una sola vez por movimiento**, igual que en el 2048 original.

src/game/logic.ts — núcleo de la función slide (simplificado)

```
for (const line of linesFor(direction)) {
  const lineTiles = line.map((c) => tileAt(tiles, c.row, c.col)).filter(Boolean);
  let target = 0, i = 0;
  while (i < lineTiles.length) {
    const current = lineTiles[i], next = lineTiles[i + 1], dest = line[target];
    if (next && next.value === current.value) {          // fusión
      nextTiles.push({ id: current.id, value: current.value * 2, row: dest.row, col: dest.col });
      ghosts.push({ ...next, row: dest.row, col: dest.col }); // ficha absorbida (animación)
      merges++; gained += current.value * 2; i += 2;
    } else {                                              // solo desplazamiento
      nextTiles.push({ ...current, row: dest.row, col: dest.col }); i += 1;
    }
    target++;
  }
}
```

Si el deslizamiento cambió el tablero, aparece una ficha nueva en una casilla vacía al azar (90 % un 2 y 10 % un 4). El estado completo (fichas, puntuación, movimientos, cargas de poderes, modo actual, avisos) se maneja con `useReducer` y las acciones `NEW_GAME`, `SLIDE`, `TOGGLE_POWER`, `CELL_PRESS`, `CLEAR_GHOSTS` y `KEEP_PLAYING`. La puntuación aumenta solo con acciones correctas, según la Tabla 2.

Tabla 2. Reglas de puntuación

Acción del jugador	Efecto
Fusionar dos fichas iguales	Suma el valor de la ficha creada (por ejemplo, 8 + 8 → **+16 puntos**).
Combo: varias fusiones en un solo deslizamiento	Bono de **+20 puntos por cada fusión extra** (3 fusiones → +40).
Alcanzar un nuevo nivel máximo desde Lanzamiento (64)	No suma puntos, pero otorga **+1 carga de cada poder** (máximo 5).
Deslizar sin fusionar	0 puntos (solo reposiciona y aparece una ficha nueva).
Usar un poder	0 puntos; no cuenta como turno ni genera ficha nueva.

Después de cada acción se evalúa el tablero: está **bloqueado** si ningún deslizamiento lo cambia, y la partida **termina** solo si está bloqueado y además no quedan cargas de Eliminar ni un intercambio útil (Mover sirve únicamente si existen al menos dos fichas iguales). Si el jugador llega al Unicornio (2048) se muestra un aviso con las opciones "Seguir jugando" o "Terminar partida". El récord se guarda en el teléfono con AsyncStorage.

4.5 Mecánicas especiales: Mover y Eliminar

Las dos mecánicas se implementaron como "poderes" con cargas limitadas, para que sean una ayuda estratégica y no eliminen el reto. Al activarlos, el tablero entra en un modo de selección: se muestra una capa de casillas táctiles encima de las fichas, con contorno punteado azul (Mover) o rojo (Eliminar), y los deslizamientos quedan desactivados.

- **Mover:** el jugador toca la ficha que quiere mover (queda resaltada con un anillo naranja) y después toca el destino. Si la casilla destino está vacía, la ficha se traslada; si está ocupada, ambas fichas **intercambian** posiciones. Tocar de nuevo la ficha seleccionada la deselecciona.
- **Eliminar:** el jugador toca cualquier ficha y esta desaparece con una animación de encogimiento. No se permite eliminar la última ficha del tablero, porque dejaría el juego sin movimientos posibles.
- Se inicia con **2 cargas** de cada poder y se gana **+1 de cada uno** al crear una ficha de un nivel nunca alcanzado a partir de Lanzamiento (64), con un máximo de 5. El botón muestra las cargas como puntos llenos.

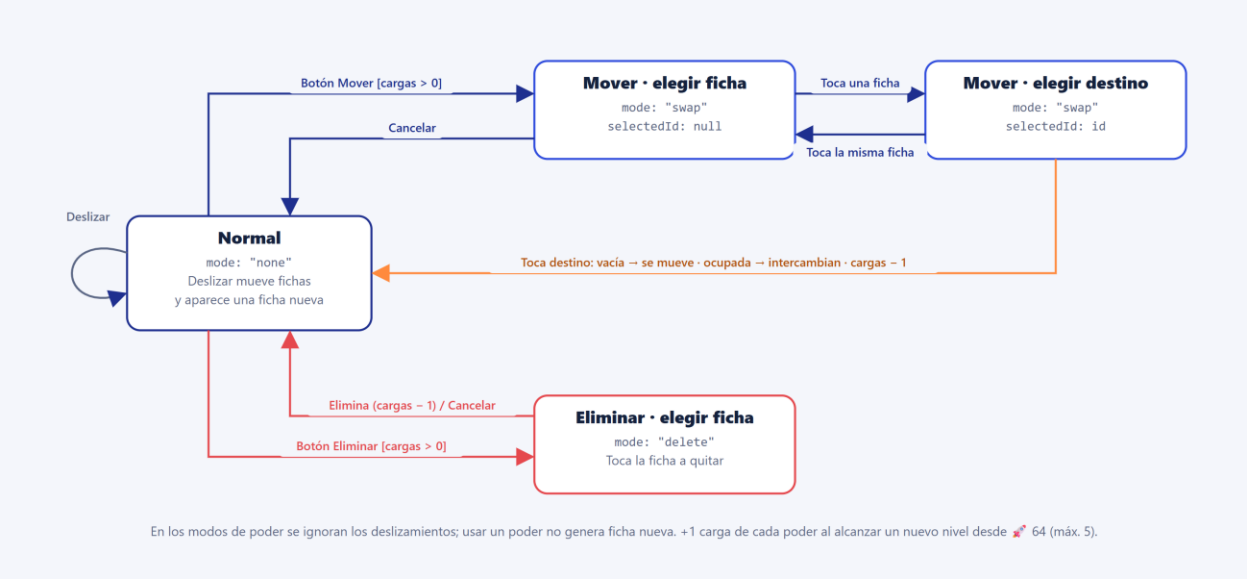


Figura 3. Diagrama de estados de los poderes Mover y Eliminar.

src/game/logic.ts — mover o intercambiar una ficha

```
export function moveOrSwap(tiles: Tile[], tileId: number, target: Cell): Tile[] {
  const source = tiles.find((tile) => tile.id === tileId);
  if (!source) return tiles;
  const occupant = tileAt(tiles, target.row, target.col);
  return tiles.map((tile) => {
    if (tile.id === source.id) return { ...tile, row: target.row, col: target.col };
    if (occupant && tile.id === occupant.id) return { ...tile, row: source.row, col: source.col };
    return tile;
  });
}
```

4.6 Gestos, animaciones y retroalimentación

Gestos. El hook `useSwipeControls` crea un `PanResponder` aplicado a toda el área de juego. Solo toma el control cuando el dedo se mueve más de 8 px (así los toques simples siguen llegando a los botones) y, al soltar, interpreta la dirección según el eje dominante si el recorrido supera 24 px. En modo de poder el `PanResponder` se desactiva. En la versión web también se aceptan las flechas del teclado y W-A-S-D, lo que facilitó las pruebas.

src/game/useControls.ts (extracto)

```
PanResponder.create({
  onStartShouldSetPanResponder: () => false,                 // no bloquea los toques
  onMoveShouldSetPanResponder: (_, g) => Math.abs(g.dx) > 8 || Math.abs(g.dy) > 8,
  onPanResponderRelease: (_, g) => {
    const absX = Math.abs(g.dx), absY = Math.abs(g.dy);
    if (Math.max(absX, absY) < MIN_SWIPE_DISTANCE) return;
    onSwipe(absX > absY ? (g.dx > 0 ? 'right' : 'left') : g.dy > 0 ? 'down' : 'up');
  },
})
```

Animaciones. Cada `TileView` mantiene un `Animated.ValueXY` con su posición y un `Animated.Value` de escala. Cuando cambian la fila o la columna, la ficha se desliza en 110 ms; las fichas nuevas aparecen con un rebote (spring) justo después del deslizamiento; al fusionarse, la ficha resultante late (escala 1 → 1.16 → 1); y al eliminarse, se encoge hasta desaparecer. En iOS y Android se usa `useNativeDriver` porque solo se animan transformaciones; en web, donde no existe ese controlador, se desactiva automáticamente.

Para que la fusión se vea natural, la ficha absorbida no se borra de inmediato: se conserva durante 220 ms como "fantasma" con el mismo `id`, se dibuja debajo de la ficha viva y se desliza hasta la casilla de destino. Además, un texto flotante "+puntos" sube sobre el marcador cada vez que se suman puntos.

Retroalimentación háptica. Con `expo-haptics` el teléfono vibra suavemente en cada fusión, con más intensidad en los combos y al usar un poder, y con patrones de notificación al alcanzar un nuevo nivel (éxito) o al bloquearse el tablero (advertencia).

4.7 Estilos reutilizables e identidad visual

Todos los valores visuales se definieron como **tokens** en `src/theme/index.ts`: colores, espaciados, radios de borde, tipografía y sombras (con `Platform.select` para usar `shadow*` en iOS, `elevation` en Android y `boxShadow` en web). Ningún componente usa colores sueltos; todos consumen estos tokens, por lo que un cambio de paleta se refleja en toda la app.

src/theme/index.ts (extracto)

```
export const colors = {
  background: '#F4F6FB', surface: '#FFFFFF', primary: '#2F4CDD', primaryDark: '#1E2F8F',
  primarySoft: '#E3E8FD', accent: '#FF8A3D', success: '#1FAF7A', danger: '#E5484D',
  text: '#14213D', textMuted: '#5B6785', board: '#C9D3EA', cell: '#DFE6F5', ...
} as const;
export const spacing = { xs: 4, sm: 8, md: 12, lg: 16, xl: 24, xxl: 32 } as const;
export const radii = { sm: 8, md: 12, lg: 18, xl: 24, pill: 999 } as const;
```

Sobre los tokens se construyeron componentes reutilizables: `Screen` (contenedor con áreas seguras y ancho máximo para tablets), `AppButton` (cuatro variantes: principal, secundario, acento y suave), `Card`, `StatBox`, `PowerButton`, `Board` y `TileView`. La identidad combina un **azul corporativo** que transmite confianza con **acentos cálidos** (naranja) y formas redondeadas que hacen la experiencia amigable, acorde al objetivo de entretener de forma sana, divertida y corporativa. Los textos generales de la app se centralizaron en `src/constants/app.ts`.

4.8 Imágenes de los recursos utilizados

Las siguientes figuras muestran los recursos gráficos del proyecto: los niveles de las fichas (emoji, nombre y colores), la paleta de colores con la iconografía de la interfaz y los íconos de la aplicación.

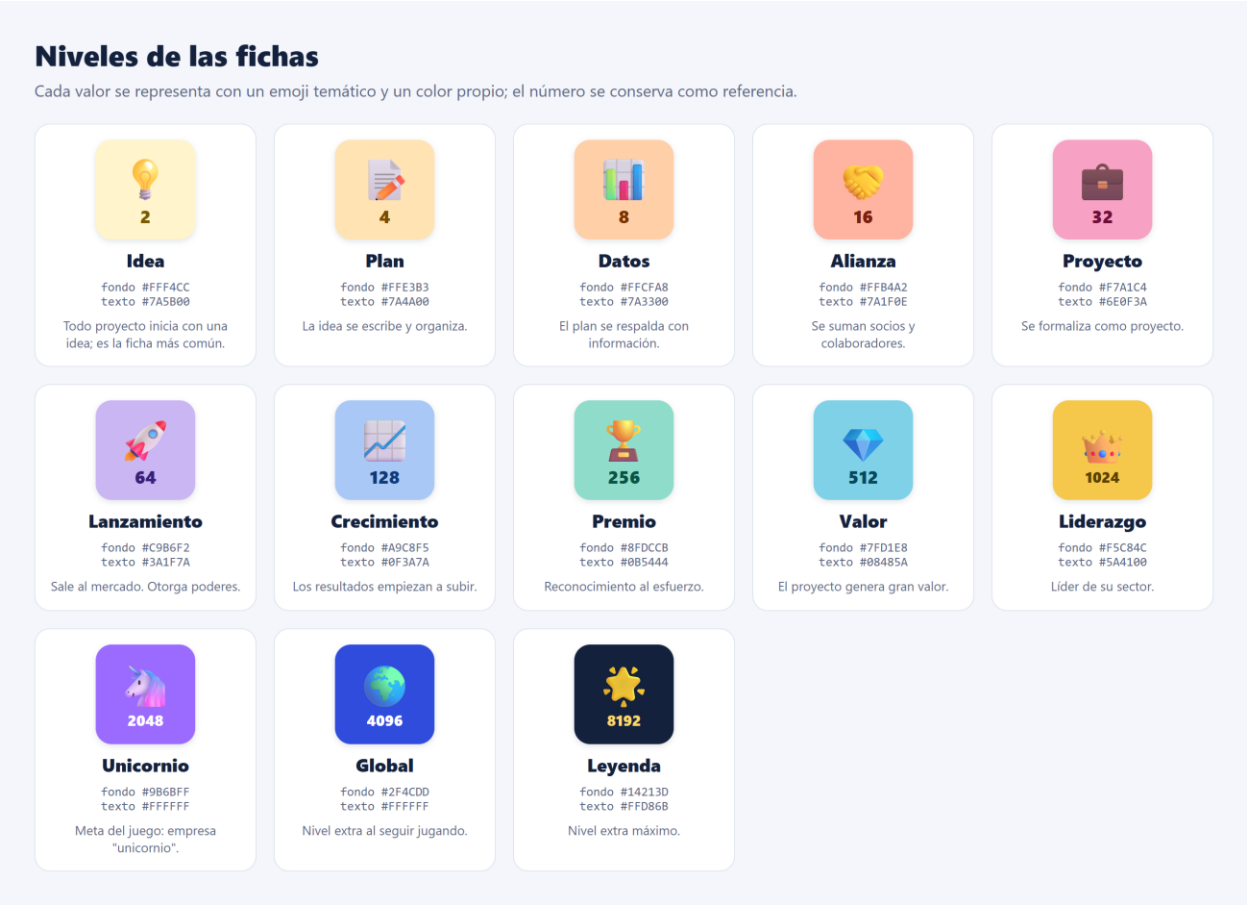


Figura 4. Niveles de las fichas: emoji, nombre, colores de fondo y texto.



Figura 5. Paleta de colores (tokens del tema) e iconografía de la interfaz.

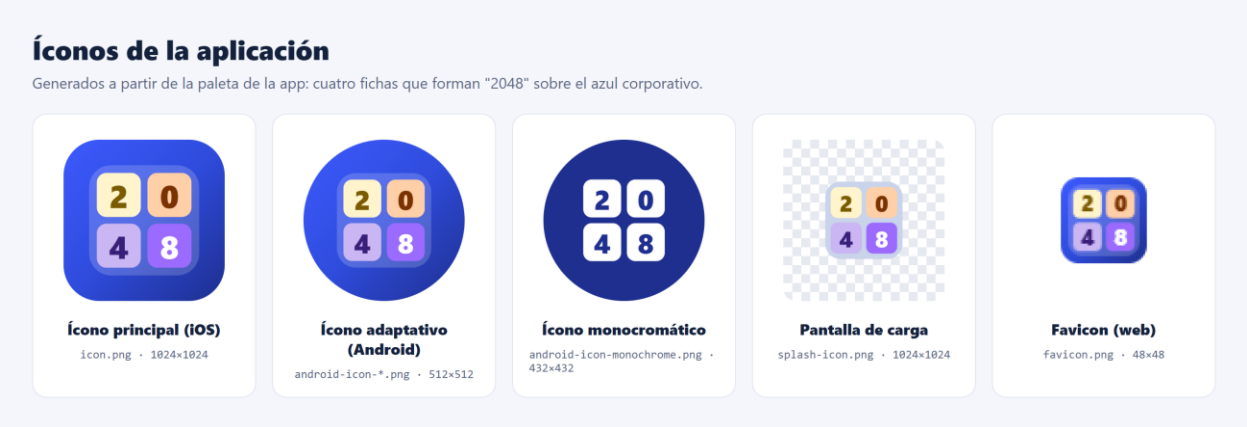


Figura 6. Íconos de la aplicación para iOS, Android, pantalla de carga y web.

Los emojis se dibujan con la fuente del sistema: Apple Color Emoji en iOS, Noto Color Emoji en Android y Segoe UI Emoji en Windows (fuente usada en las figuras de este reporte). Por eso su aspecto varía ligeramente entre dispositivos, pero su significado se mantiene.

4.9 Explicación y justificación de los gráficos de las celdas

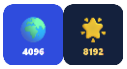
Se eligió una narrativa de **crecimiento empresarial, "De una idea a un unicornio"**: cada fusión representa un paso en la evolución de un proyecto, desde una Idea (2) hasta una empresa Unicornio (2048), término que se usa para las startups valuadas en más de mil millones de dólares. Esto le da sentido a la progresión de niveles y encaja con un entretenimiento corporativo y sano: el jugador "hace crecer" un proyecto en lugar de solo sumar números.

- **Triple codificación (forma + color + número).** Cada nivel se distingue por su emoji, por su color de fondo y por el valor numérico. Si una persona no distingue bien ciertos colores, todavía puede

identificar la ficha por el emoji o el número; y el número sigue siendo necesario para planear fusiones.

- **Progresión de color con significado.** Los niveles bajos usan tonos pastel cálidos (amarillo, durazno, salmón, rosa), los intermedios pasan a tonos fríos (lila, azul, verde agua, cian), el 1024 es dorado y la meta 2048 usa un violeta saturado con texto blanco para que destaque en el tablero. Los niveles extra (4096 y 8192) usan el azul de marca y el azul marino.
- **Legibilidad.** El texto de cada ficha usa una versión oscura del mismo tono de fondo, lo que asegura buen contraste sin romper la armonía de color.
- **Emojis en lugar de imágenes.** No requieren archivos, no aumentan el tamaño de la app, se escalan sin perder calidad a cualquier tamaño de pantalla y son reconocibles en todo el mundo. La desventaja es que su dibujo depende del sistema operativo; se mitigó con el color y el número.
- **Coherencia temática.** El poder Mover usa el símbolo de reordenar (flechas cruzadas) y Eliminar el de un bote de basura, íconos conocidos que no requieren explicación.

Tabla 3. Significado de cada nivel de ficha

Valor	Gráfico	Nivel	Justificación
2		Idea	Todo proyecto comienza con una idea; es la ficha más frecuente.
4		Plan	La idea se escribe y se organiza.
8		Datos	El plan se respalda con información y métricas.
16		Alianza	Se suman socios y colaboradores.
32		Proyecto	La iniciativa se formaliza como proyecto.
64		Lanzamiento	Sale al mercado; primer nivel que otorga poderes.
128		Crecimiento	Los resultados empiezan a subir.
256		Premio	Reconocimiento al esfuerzo del equipo.
512		Valor	El proyecto genera gran valor.
1024		Liderazgo	Líder de su sector; color dorado.
2048		Unicornio	Meta del juego; violeta saturado para destacar.
4096 / 8192		Global / Leyenda	Niveles extra para quien sigue jugando.

4.10 Capturas de las pantallas de la aplicación

Las capturas se tomaron de la aplicación en tamaño de teléfono (390 × 844 puntos) durante una partida real jugada de forma automatizada hasta el final. A continuación se agrupan por pantalla.

Pantalla de presentación

Muestra el nombre de la app, su propósito, las instrucciones de juego, la explicación de los poderes y la tabla de niveles. Los botones "Jugar" y "Créditos" permanecen fijos en la parte inferior.



Figura 7. Presentación: nombre y propósito.



Figura 8. Presentación: cómo jugar.



Figura 9. Presentación: poderes y niveles.

Pantalla de juego

Incluye el marcador (puntos, récord y movimientos), el nivel actual, el tablero, una barra de avisos y los botones de poderes con sus cargas.



Figura 10. Juego: inicio de partida.

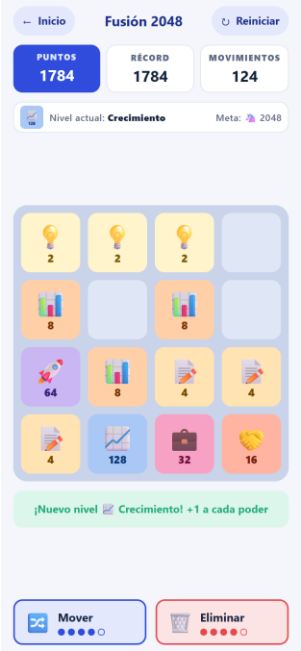


Figura 11. Juego: aviso de nuevo nivel y recompensa de poderes.

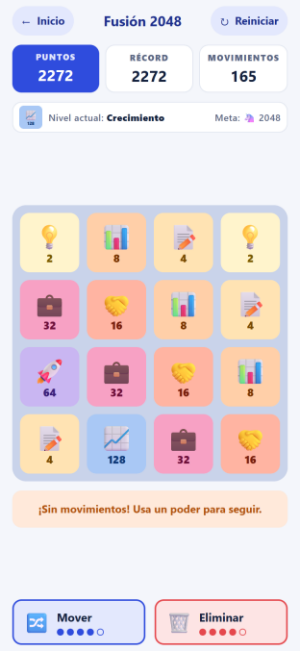


Figura 12. Juego: tablero lleno; se sugiere usar un poder.

Poder Mover

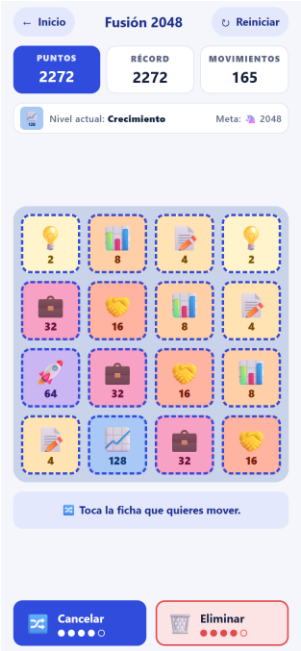


Figura 13. Mover: fichas seleccionables.

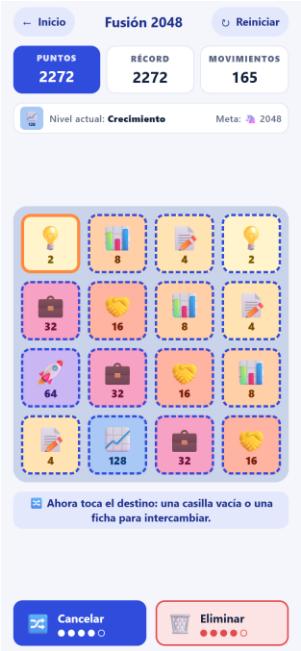


Figura 14. Mover: ficha elegida (anillo naranja) y destinos.

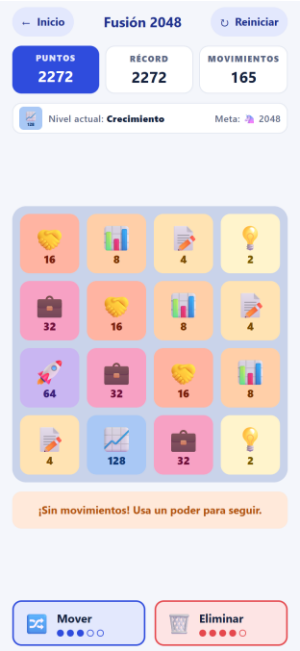


Figura 15. Mover: las fichas de las esquinas intercambiadas.

Poder Eliminar y fin de los movimientos



Figura 16. Eliminar: elegir la ficha a quitar.

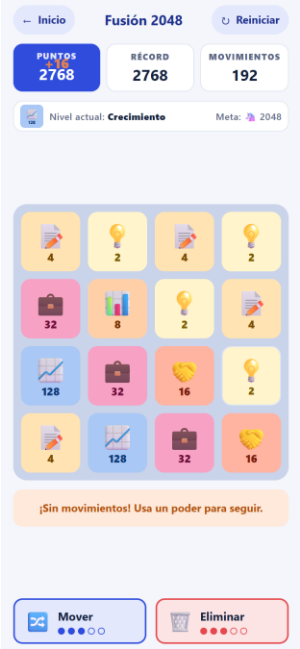


Figura 17. Tablero bloqueado con poderes disponibles.

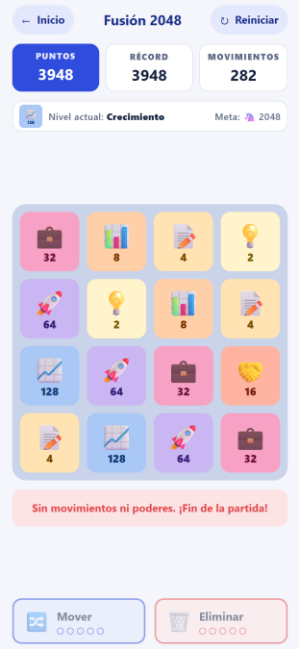


Figura 18. Sin movimientos ni poderes: fin de la partida.

Pantallas de Game Over y créditos



Figura 19. Game Over: puntuación final, nuevo récord y estadísticas.



Figura 20. Créditos: equipo y concepto.



Figura 21. Créditos: tecnologías y gráficos.

4.11 Ejecución en un dispositivo físico con Expo Go

1. Instalar **Expo Go** desde la App Store o Google Play (debe corresponder al SDK 57).
2. Conectar la computadora y el teléfono a la misma red Wi-Fi.
3. En la carpeta del proyecto ejecutar `npx expo start` (en PowerShell: `npx.cmd expo start`).
4. Escanear el código QR: en iOS con la app Cámara; en Android desde Expo Go. En iOS se debe aceptar el permiso de "Red local".
5. Si la red bloquea la conexión entre dispositivos, usar `npx expo start --tunnel`.

5. Análisis de resultados

La aplicación cumple con los requisitos planteados: cuenta con las cuatro pantallas navegadas con Expo Router, implementa la mecánica de fusión del 2048 con puntuación por acciones correctas, representa cada nivel con un gráfico propio, incluye las mecánicas especiales Mover y Eliminar, y utiliza un sistema de estilos reutilizables. A continuación se describen las pruebas realizadas y los problemas encontrados.

5.1 Pruebas realizadas

Tabla 4. Pruebas realizadas y resultados

Prueba	Método / herramienta	Resultado
Pruebas unitarias de la lógica	Script con `node:assert`: deslizar en las 4 direcciones, una fusión por ficha y movimiento, conservación de identificadores, detección de bloqueo, mover/intercambiar y aparición de fichas.	Todas aprobadas.
Simulación de partidas completas	Script que juega 200 partidas con movimientos aleatorios usando el mismo reducer de la app y, al bloquearse, usa los poderes hasta agotarlos.	200 de 200 partidas terminaron correctamente; 1,703 bloqueos resueltos con poderes; 452 recompensas de cargas; sin fichas duplicadas en posición ni en identificador.
Verificación de tipos	`npx tsc --noEmit`	0 errores.
Análisis estático	`npx expo lint` (ESLint con eslint-config-expo)	0 problemas (después de corregir 24 detectados; ver 5.2).
Compatibilidad de dependencias	`npx expo-doctor`	21 de 21 verificaciones aprobadas.
Empaquetado nativo	`npx expo export --platform android --platform ios`	Paquetes Hermes generados: Android 2.8 MB, iOS 2.4 MB.
Prueba funcional de la interfaz	Versión web en tamaños 375×812 y 360×640; recorrido automatizado en Chrome que jugó una partida completa (282 movimientos, 3,948 puntos, ficha máxima 128 "Crecimiento", 8 poderes usados).	Todas las pantallas, gestos, poderes y la navegación funcionaron; ver capturas de la sección 4.10.
Prueba en dispositivo físico	Expo Go (SDK 57) escaneando el QR de `npx expo start`.	La app carga y responde a deslizamientos, animaciones, vibración y navegación.

5.2 Problemas encontrados y soluciones

Tabla 5. Problemas encontrados durante el desarrollo y su solución

Problema	Causa	Solución
`npx expo start` fallaba en PowerShell con "la ejecución de scripts está deshabilitada".	La política de ejecución de PowerShell bloquea el script `npx.ps1`.	Usar `npx.cmd expo start` (o cmd / Git Bash), o permitir scripts locales con `Set-ExecutionPolicy -Scope CurrentUser RemoteSigned`.
El iPhone no abría el proyecto desde Expo Go.	En iOS el QR se escanea con la Cámara y Expo Go necesita el permiso de "Red local"; algunas redes bloquean la conexión.	Escanear con la Cámara, activar Ajustes → Expo Go → Red local, y usar `--tunnel` en redes restringidas.
`SafeAreaView` de React Native marcado como obsoleto.	En React Native 0.86 el componente está en desuso.	Se usó `SafeAreaView` de `react-native-safe-area-context`.
Al fusionar, la ficha absorbida desaparecía de golpe.	Se eliminaba del estado en el mismo instante del movimiento.	Fichas "fantasma" que conservan su `id` durante 220 ms y se deslizan hasta el destino antes de borrarse.
Deslizar a la derecha podía activar el gesto "atrás" de iOS; el botón atrás de Android salía de la partida.	Conflicto entre los gestos del navegador y los del tablero.	`gestureEnabled: false` en la pantalla de juego y `BackHandler` con diálogo de confirmación.
En modo de poder, los toques en el tablero podían interpretarse como deslizamientos.	El `PanResponder` seguía activo sobre el área de juego.	Se desactiva el `PanResponder` en modo de poder y se dibuja una capa de casillas `Pressable` encima de las fichas.
La partida terminaba aunque el jugador aún tuviera poderes.	Solo se evaluaba si existían deslizamientos posibles.	Se separaron los estados "bloqueado" y "terminado": termina solo si no quedan cargas de Eliminar ni un intercambio útil.
Eliminar la última ficha dejaba un tablero vacío sin movimientos.	Sin fichas no hay deslizamiento válido ni ficha nueva.	Se impide eliminar la última ficha y se muestra un aviso.
El borde de la ficha seleccionada no se veía completo.	El contenido de la ficha cubría el borde del contenedor.	Se dibuja un anillo superpuesto con posición absoluta.
24 errores de lint `react-hooks/refs`.	Las reglas del React Compiler no permiten leer `ref.current` durante el render (`useRef(new Animated.Value()).current`).	Valores animados creados con `useState(() => new Animated.Value())` y `PanResponder` memorizado según si está habilitado.
Advertencias de animación en la versión web.	`useNativeDriver` no existe en web.	Bandera `USE_NATIVE_DRIVER = Platform.OS !== 'web'`.
Nombres de nivel cortados ("Lanzami…", "Crecimi…").	Ancho insuficiente en la leyenda y en la caja de estadísticas.	Se ajustó el ancho y el tamaño de letra; en Game Over el nombre del nivel se muestra bajo la ficha.
Un emoji compuesto (persona con computadora) no se dibujaba en Windows.	Las secuencias de emojis unidas (ZWJ) no tienen soporte en todas las fuentes.	Se reemplazó por un emoji simple (dos personas).
En el emulador táctil del navegador los botones dejaban de responder tras un arrastre.	Comportamiento del emulador de touch, no de la app.	Se comprobó sin emulación: funcionan correctamente.

Problema	Causa	Solución
Aparecía un botón flotante de desarrollo en las capturas web.	Indicador del servidor de desarrollo de Expo.	Las capturas se tomaron de la versión de producción (`npx expo export -p web` + `npx expo serve`).
expo-doctor detectó versiones de parche desactualizadas.	Expo publicó parches del SDK 57 después de crear el proyecto.	`npx expo install --fix`; nueva verificación 21/21.

En resumen, la mayoría de los problemas surgieron en la **interacción** (gestos que compiten entre sí, animaciones que deben reflejar cambios de estado instantáneos) y en el **entorno** (políticas del sistema, permisos de red, versiones del SDK). Separar la lógica en funciones puras permitió probar las reglas del juego con miles de movimientos simulados, independientemente de la interfaz.

6. Mejoras a futuro

6.1 Tablero de puntuaciones en línea mediante una API web segura

Se propone, de forma conceptual, un sistema para subir las puntuaciones finales a un tablero global. El principio central es **no confiar en el cliente**: una puntuación calculada en el teléfono puede alterarse modificando la app, interceptando el tráfico o enviando peticiones falsas directamente a la API. Por eso el servidor debe poder **verificar** cada puntuación en lugar de solo recibirla.

Amenazas consideradas

- **Suplantación de identidad:** enviar puntuaciones a nombre de otro jugador.
- **Suplantación del cliente:** scripts o bots que llaman a la API sin usar la app real.
- **Puntuación falsificada:** modificar el número enviado o el código del juego.
- **Repetición (replay):** reenviar una petición válida capturada para duplicar un resultado.
- **Intermediario (MITM):** alterar los datos en tránsito.
- **Abuso masivo:** muchas cuentas o peticiones para saturar el tablero.

Flujo propuesto

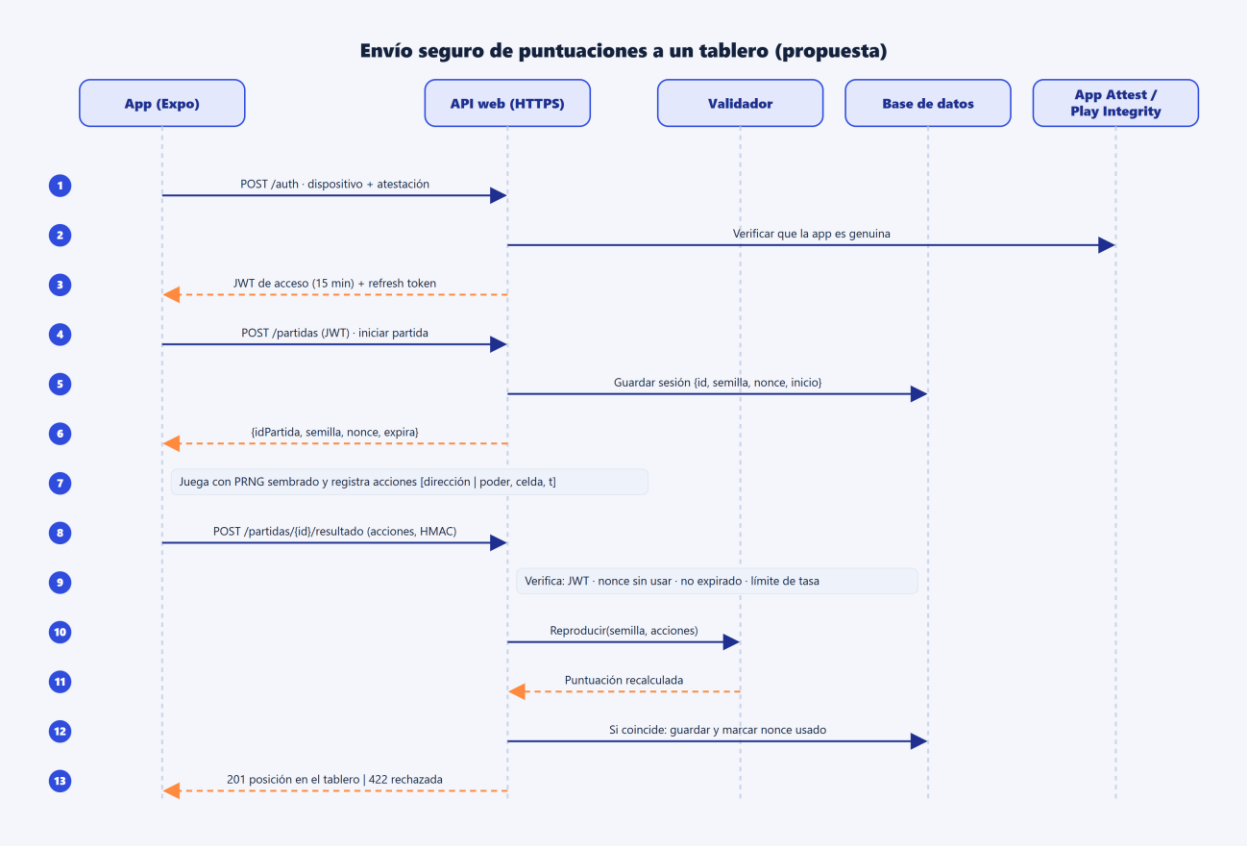


Figura 22. Secuencia propuesta para el envío y la validación de puntuaciones.

1. **Autenticación y atestación.** La app inicia sesión (cuenta de Apple/Google mediante OAuth u otra cuenta anónima ligada al dispositivo) y adjunta un token de **App Attest** (iOS) o **Play Integrity** (Android), que demuestra que la petición viene de la app original sin modificar y de un dispositivo real. El servidor responde con un **JWT** de corta duración (15 min) y un token de renovación, guardados en el almacenamiento seguro del sistema (Keychain/Keystore).
2. **Inicio de partida emitido por el servidor.** Antes de jugar, la app solicita `POST /partidas`. El servidor crea una sesión y devuelve un identificador, una **semilla** aleatoria, un **nonce** de un solo uso y una fecha de expiración.
3. **Juego determinista y registro de acciones.** La app usa la semilla en un generador pseudoaleatorio (por ejemplo, mulberry32) para decidir dónde aparecen las fichas, y registra cada acción: dirección de deslizamiento o poder usado, celdas y marca de tiempo. La función `spawnTile` del proyecto ya recibe el generador como parámetro, por lo que el cambio es mínimo.
4. **Envío del resultado.** Al terminar, la app envía las acciones, la puntuación declarada y una firma HMAC calculada con una clave de sesión entregada por el servidor (no una clave fija dentro de la app, que podría extraerse).
5. **Validación en el servidor.** Se verifica el JWT, que el nonce no se haya usado y que la sesión no haya expirado; se aplican límites de tasa por usuario, dispositivo e IP; y se valida el formato y el tamaño de los datos. Después, el servidor **reproduce la partida** con la misma semilla y las mismas acciones usando el mismo código de `logic.ts` (TypeScript puro, ejecutable en Node.js). La puntuación recalculada debe coincidir con la declarada y los tiempos entre movimientos deben ser plausibles.
6. **Registro.** Si todo coincide, se guarda la puntuación, se marca el nonce como usado y se responde con la posición en el tablero (`201`); si no, se rechaza (`422`) y se registra para auditoría. Solo el servidor escribe en la base de datos.

Tabla 6. Amenazas y contramedidas del tablero de puntuaciones

Amenaza	Contramedida
Suplantación de identidad	Inicio de sesión OAuth; JWT firmado por el servidor, de corta duración, ligado al usuario que inició la partida.
Suplantación del cliente / bots	App Attest / Play Integrity; límites de tasa; detección de patrones anómalos.
Puntuación falsificada	Reproducción determinista de la partida en el servidor; la puntuación se recalcula, no se acepta la del cliente.
Repetición (replay)	Nonce de un solo uso con expiración; la sesión solo acepta un resultado.
Intermediario (MITM)	HTTPS con TLS 1.3 y HSTS; opcionalmente fijado de certificado; firma HMAC del cuerpo.
Abuso masivo	Límites por usuario, dispositivo e IP; revisión de puntuaciones atípicas (por encima del percentil 99).

Ejemplo de envío de resultado

```
POST /partidas/7f3c…/resultado          Authorization: Bearer <JWT>
{
  "nonce": "b81e…",
  "puntuacion": 3948,
  "acciones": [ { "t": 412, "tipo": "slide", "dir": "down" },
                { "t": 958, "tipo": "swap", "de": [0,0], "a": [3,3] },
                { "t": 1510, "tipo": "delete", "celda": [2,1] }, ... ],
  "firma": "HMAC-SHA256(claveDeSesion, cuerpo)"
}
```

Ningún sistema es invulnerable: un atacante decidido podría programar un bot que juegue de verdad a través de la app. Sin embargo, el diseño propuesto obliga a jugar partidas reales y verificables, con tiempos humanos y desde una app genuina, lo que eleva mucho el costo del fraude. Hay que considerar que la atestación requiere módulos nativos, por lo que esta mejora necesitaría una compilación de desarrollo (EAS Build) en lugar de Expo Go.

6.2 Otras mejoras

- Modo oscuro aprovechando que todos los colores ya son tokens centralizados.
- Efectos de sonido y música opcionales, con control de volumen.
- Botón de "deshacer" como tercer poder, y tableros de 5×5 o 6×6.
- Migrar las animaciones a Reanimated y los gestos a Gesture Handler para ejecutarlos en el hilo de interfaz.
- Tutorial interactivo en la primera partida y estadísticas históricas del jugador.
- Publicación en las tiendas con EAS Build y actualizaciones con EAS Update.

7. Conclusiones

Se desarrolló **Fusión 2048**, un juego tipo 2048 con temática de crecimiento empresarial, cuatro pantallas navegadas con Expo Router, gestos de deslizamiento, animaciones, vibración háptica, récord persistente y dos mecánicas especiales (Mover y Eliminar) que agregan decisiones estratégicas al juego clásico. La aplicación se ejecuta en un dispositivo físico mediante Expo Go sin necesidad de compilar código nativo.

7.1 Ventajas de React Native y Expo

- **Un solo código para varias plataformas.** El mismo TypeScript funciona en iOS, Android y web (útil para pruebas rápidas y capturas).
- **Ciclo de desarrollo rápido.** Expo Go y la recarga en caliente permiten ver los cambios en el teléfono en segundos; `expo install` y `expo-doctor` evitan conflictos de versiones.
- **Navegación sencilla.** Con Expo Router, la estructura de carpetas es la navegación; pasar parámetros, reemplazar pantallas o volver al inicio requiere una sola línea.
- **Estado predecible.** Con `useReducer`, todas las reglas del juego se concentran en una función pura que se puede probar por separado (incluso simulando cientos de partidas).
- **Interfaz rica y accesible.** Textos, botones, tarjetas, avisos, diálogos y etiquetas de accesibilidad se resuelven con componentes estándar y estilos reutilizables.

7.2 Dificultades

- **Animar un modelo declarativo.** React describe el estado final, pero un juego necesita mostrar la transición (deslizar, fusionar, desaparecer). Fue necesario usar la API `Animated` de forma imperativa, identificadores estables y fichas "fantasma" para conectar ambos mundos.
- **Gestos que compiten.** Los deslizamientos del tablero chocan con el gesto "atrás" de iOS, el botón atrás de Android y los toques de los botones; hubo que coordinar el sistema de respondedores con la navegación.
- **Hilo de JavaScript.** La lógica y las animaciones corren por defecto en el hilo de JavaScript; solo las transformaciones y la opacidad pueden delegarse al hilo nativo. Para animaciones más complejas se requerirían bibliotecas adicionales (Reanimated, Gesture Handler).
- **Limitaciones de Expo Go y cambios frecuentes.** Solo se pueden usar los módulos nativos incluidos en Expo Go, y cada versión del SDK depreca APIs (como `SafeAreaView`) o agrega reglas nuevas (React Compiler), lo que obliga a consultar documentación actualizada.
- **Diferencias entre plataformas.** Los emojis, las sombras y algunos comportamientos varían entre iOS, Android y web.

7.3 Contraste con motores de videojuegos: objetos de juego y ECS

Los motores de videojuegos como Unity o Godot organizan el juego con un **modelo de objetos de juego**: cada elemento (por ejemplo, una ficha) es un objeto en una escena con componentes (sprite, transformación, script) y un **bucle de juego** llama a su actualización en cada cuadro, unas 60 veces por

segundo. Las animaciones, la física y el dibujo en 2D/3D vienen integrados y se configuran en un editor visual.

El patrón **Entity Component System (ECS)**, usado por Unity DOTS, Bevy o Flecs, lleva esta idea a un diseño orientado a datos: las **entidades** son solo identificadores, los **componentes** son datos puros (posición, valor, animación) y los **sistemas** son funciones que procesan en cada cuadro todas las entidades que tienen ciertos componentes. Esto aprovecha mejor la memoria y escala a miles de entidades.

Curiosamente, nuestra app tiene paralelos con ECS: cada ficha `{ id, value, row, col }` es como una entidad con componentes de datos, y el reducer funciona como un "sistema" que transforma todo el tablero. La diferencia principal es el modelo de actualización: **React es dirigido por eventos** (solo vuelve a dibujar cuando cambia el estado), mientras que los motores **actualizan continuamente** en cada cuadro.

Tabla 7. Comparación entre React Native/Expo y los modelos de los motores de juego

Aspecto	React Native + Expo	Objetos de juego (Unity / Godot)	ECS (DOTS / Bevy)
Actualización	Por eventos: se vuelve a dibujar al cambiar el estado.	Bucle de juego: Update() en cada cuadro.	Sistemas que se ejecutan en cada cuadro.
Una ficha es…	Un objeto de estado + un componente visual.	Un GameObject con sprite y script.	Una entidad con componentes de datos.
Animación	API Animated imperativa sobre UI declarativa.	Animador y tweens integrados.	Componentes de animación procesados por un sistema.
Interfaz y menús	Excelente: texto, navegación, accesibilidad, formularios.	Posible, pero más laboriosa.	Generalmente la más laboriosa.
Muchas entidades / física	Limitado.	Bueno.	Excelente (orientado a datos).
Iteración en móvil	Muy rápida con Expo Go.	Requiere compilar.	Requiere compilar.

Para un juego **por turnos** como 2048, donde el estado solo cambia con la acción del jugador y la interfaz (menús, instrucciones, créditos, récords) pesa tanto como el tablero, React Native con Expo resultó una opción adecuada y productiva. Para juegos **en tiempo real** con física, muchos objetos en pantalla o gráficos intensivos, un motor de videojuegos o una arquitectura ECS sería más apropiada. Como balance, la práctica mostró que React Native y Expo permiten construir interfaces móviles interactivas de buena calidad con rapidez, siempre que se entiendan sus límites al combinar navegación, estado, gestos y animaciones.

8. Referencias

Expo. (2026). *Expo SDK 57 documentation*. https://docs.expo.dev/versions/v57.0.0/

Expo. (2026). *Introduction to Expo Router*. https://docs.expo.dev/router/introduction/

Expo. (2026). *Expo Go*. https://docs.expo.dev/get-started/expo-go/

Meta Platforms. (2026). *React Native: Animations*. https://reactnative.dev/docs/animations

Meta Platforms. (2026). *React Native: PanResponder y Gesture Responder System*. https://reactnative.dev/docs/panresponder

Meta Platforms. (2026). *React: useReducer*. https://react.dev/reference/react/useReducer

Cirulli, G. (2014). *2048* [Código fuente]. https://github.com/gabrielecirulli/2048

Wikipedia. *2048 (videojuego)*. https://es.wikipedia.org/wiki/2048_(videojuego)

Wikipedia. *Tile-matching video game*. https://en.wikipedia.org/wiki/Tile-matching_video_game

Wikipedia. *Motor de videojuego*. https://es.wikipedia.org/wiki/Motor_de_videojuego

Wikipedia. *Entity component system*. https://en.wikipedia.org/wiki/Entity_component_system

Unity Technologies. *Entities (DOTS) documentation*. https://docs.unity3d.com/Packages/com.unity.entities@latest

Apple. *Establishing your app's integrity (App Attest)*. https://developer.apple.com/documentation/devicecheck/establishing-your-app-s-integrity

Google. *Play Integrity API*. https://developer.android.com/google/play/integrity

OWASP. *Mobile Application Security Verification Standard (MASVS)*. https://mas.owasp.org/MASVS/

Jones, M., Bradley, J. y Sakimura, N. (2015). *RFC 7519: JSON Web Token (JWT)*. IETF. https://www.rfc-editor.org/rfc/rfc7519